



Escola Politécnica de Enxeñaría de Ferrol

UNIVERSIDADE DA CORUÑA



Introducción al Aprendizaje Automático CURSO 2025/2026

Preparación de los datos y clasificación supervisada

M. U. en Ingeniería Industrial

AUTORES: ÁLVARO NAVEIRA CAAVEIRO
FRANCISCO RODRÍGUEZ GAVÍN

TUTORES: ALMA MARÍA MALLO CASDELO
FRANCISCO JAVIER BELLAS BOUZA

FECHA: MARZO DE 2026

Índice

1	Introducción	2
2	Descripción del conjunto de datos	2
3	Descripción de las pruebas realizadas	2
3.1	Proceso completo	3
3.1.1	Eliminar filas con información incompleta	3
3.1.2	Transformación de los valores de texto a número	3
3.1.3	División del conjunto de datos en datos de entrada y salida	4
3.1.4	División del conjunto de datos en datos de entrenamiento y prueba	5

1. Introducción

El presente trabajo tiene el objetivo de analizar el funcionamiento de dos técnicas de aprendizaje supervisado sobre un conjunto de datos etiquetados, de modo que se pueda realizar una caracterización objetiva de su funcionamiento en base a los resultados obtenidos. Para ello, primero se escogerá un conjunto de datos que sea válido para utilizarlo en tareas de clasificación, luego se le aplicarán las distintas técnicas de preprocesado que se consideren oportunas para finalmente, aplicar las técnicas de aprendizaje automático y realizar una comparación de sus resultados.

Para poder realizar este análisis, se implementará un programa en Python utilizando la librería *scikit-learn* para aplicar las técnicas de preprocesado de los datos y de aprendizaje automático. Además, se emplearán otras librerías como *Pandas* o *Numpy*, necesarias para la correcta lectura y manejo de los datos.

2. Descripción del conjunto de datos

La elección del conjunto de datos con el que se vaya a trabajar representa el primer paso de este proyecto. Como el objetivo es emplear conjuntos de datos reales, se han consultado distintas opciones en plataformas web como Kaggle o UCI y finalmente se ha escogido una base de datos que permite clasificar las setas en dos tipos: comestibles o venenosas.

3. Descripción de las pruebas realizadas

Una vez hemos definido el conjunto de datos de forma completa, este se utilizará para realizar predicciones de si una seta es comestible o venenosa pero, para conseguir esto, se pueden aplicar diferentes técnicas de preprocesado y algoritmos de clasificación que puedan mejorar la eficacia y eficiencia del modelo a la hora de predecir este resultado.

Por lo tanto, las pruebas a realizar serán las siguientes:

1. Proceso completo, utilizando todas las técnicas de preprocesado
2. Utilizando únicamente aquellos procesos de preprocesado estrictamente necesarios
3. Realizando todo preprocesado a excepción de PCA y normalizado
4. Realizando todo preprocesado a excepción de normalizado

A parte de estas pruebas, también será necesario determinar ciertas características de los algoritmos de clasificación, en el caso del K-NN será necesario determinar el número de vecinos a utilizar y en el caso de SVM será necesario determinar los valores de C y que tipo de kernel es más adecuado para el conjunto de datos utilizado.

3.1. Proceso completo

Una vez definido que procesos son los que se quieren llevar a cabo, se decidió comenzar aplicando todas las técnicas de preprocesado conocidas, para posteriormente poder decidir, en función de los resultados obtenidos, si es necesario aplicar todas estas técnicas utilizadas y cuales realmente son necesarias.

3.1.1. Eliminar filas con información incompleta

Para ello comenzaremos eliminando todas aquellas filas que tengan algún valor nulo o desconocido. Esto nos ayudará a reducir el tamaño del conjunto, que es bastante extenso y a reducir tiempos de entrenamiento a la vez que se asegura la fiabilidad de los datos del mismo, este será el código completo empleado:

```
1 df_orig=pd.read_csv('mushrooms1.csv', na_values=["?"])
2 df_orig.head()
3
4
5 df_orig.shape
6 df_orig.isnull().sum()
7 df=df_orig.dropna(axis=0)
8 df.shape
9 df.isnull().sum()
```

Listing 1: Eliminar filas con información incompleta

Utilizando este código se eliminarán todas las filas con valores nulos, lo que nos deja con un total de 5936 filas, eliminando un total de 2481 filas.

3.1.2. Transformación de los valores de texto a número

Un problema que presenta este conjunto de datos es que todos sus valores están en formato texto, esto imposibilita realizar todo el estudio sobre este conjunto de datos. Para evitar esto, se pasan todos los valores presentes en el conjunto de datos de texto a número mediante la siguiente técnica de preprocesado:

```
1 encoder = preprocessing.OrdinalEncoder(dtype=int)
2 df_encoded = encoder.fit_transform(df[['poisonous', "cap-
3 shape","cap-surface","cap-color","bruises","odor",
4 "gill-attachment","gill-spacing","gill-size","gill-color","
5 stalk-shape","stalk-root",
6 "stalk-surface-above-ring","stalk-surface-below-ring","stalk-
7 color-above-ring",
8 "stalk-color-below-ring","veil-type","veil-color","ring-
9 number","ring-type",
10 "spore-print-color","population","habitat"]])
encoder.categories_
df_encoded
```

```
11 df["poisonous"] = df_encoded[:,0]
12 df["cap-shape"] = df_encoded[:,1]
13 df["cap-surface"] = df_encoded[:,2]
14 df["cap-color"] = df_encoded[:,3]
15 df["bruises"] = df_encoded[:,4]
16 df["odor"] = df_encoded[:,5]
17 df["gill-attachment"] = df_encoded[:,6]
18 df["gill-spacing"] = df_encoded[:,7]
19 df["gill-size"] = df_encoded[:,8]
20 df["gill-color"] = df_encoded[:,9]
21 df["stalk-shape"] = df_encoded[:,10]
22 df["stalk-root"] = df_encoded[:,11]
23 df["stalk-surface-above-ring"] = df_encoded[:,12]
24 df["stalk-surface-below-ring"] = df_encoded[:,13]
25 df["stalk-color-above-ring"] = df_encoded[:,14]
26 df["stalk-color-below-ring"] = df_encoded[:,15]
27 df["veil-type"] = df_encoded[:,16]
28 df["veil-color"] = df_encoded[:,17]
29 df["ring-number"] = df_encoded[:,18]
30 df["ring-type"] = df_encoded[:,19]
31 df["spore-print-color"] = df_encoded[:,20]
32 df["population"] = df_encoded[:,21]
33 df["habitat"] = df_encoded[:,22]
34 df.head()
```

Listing 2: Transformación de los valores de texto a número

Al ejecutar este código se sustituyen los valores que presenta el conjunto de datos en formato texto a la hora de clasificar los tipos de setas por números enteros, lo que nos permitirá trabajar con estos datos más adelante sin problema.

3.1.3. División del conjunto de datos en datos de entrada y salida

Una vez tenemos un conjunto de datos con el que podemos trabajar sin problema, se procederá a definir cuales son los datos de entrada y salida, para ello se seleccionarán las columnas que utilizaremos como entradas y las convertimos en un array de NumPy, para poder utilizarlo con los algoritmos de scikit learn, con las columnas que utilizaremos como salidas se hará lo mismo, utilizando el siguiente código:

```
1 df.columns
2 feature_df = df[df.columns[1:df.shape[1]]]
3 # Usando DataFrame Pandas
4 X = feature_df
5 X_readed = np.asarray(feature_df)
6 X_readed[0:5]
7 feature_df.shape
8 y_readed = np.asarray(df['poisonous'])
9 y_readed[0:5]
```

Listing 3: División del conjunto de datos en datos de entrada y salida

Por lo tanto, la columna "Poisonous" será nuestra salida, el valor que queremos predecir, y el resto de columnas constituirán las entradas, valores a partir de los cuales se realizará la predicción.

3.1.4. División del conjunto de datos en datos de entrenamiento y prueba

A la hora de entrenar un modelo, este necesita una cantidad de datos que serán de entrenamiento, los cuales estudiará y donde tratará de encontrar patrones para que, posteriormente, pueda realizar las predicciones sobre los datos de prueba. Como ya se conocen los valores de estos datos antes de que el modelo realice las predicciones, esto nos puede permitir ver cuanta precisión tiene el modelo de cara a su uso en otros ámbitos. El código empleado para realizar esta clasificación es el siguiente:

```
1 X_train, X_test, y_train, y_test = train_test_split(  
2     X_readed, y_readed, random_state = 1)  
3 print ('Train set:', X_train.shape, y_train.shape)  
4 print ('Test set:', X_test.shape, y_test.shape)
```

Listing 4: División del conjunto de datos en datos de entrenamiento y prueba

En "train_test_split" se podría añadir un valor ("test_size") que permite modificar el tamaño de cada división según se considere, por defecto este valor es de 75 % de datos de entrenamiento y un 25 % de datos de prueba, dado que nuestro conjunto de datos es bastante grande para lo que se consideraba en la práctica, se ha decidido que esta proporción es adecuada, siendo finalmente 4452 filas de datos de entrenamiento y 1484 filas de datos de prueba, a falta de terminar con el proceso de preprocesado.